

Neural Networks

(# Back-Propagation Training Rules & Algorithm
For
Multi-layer Neural network #)

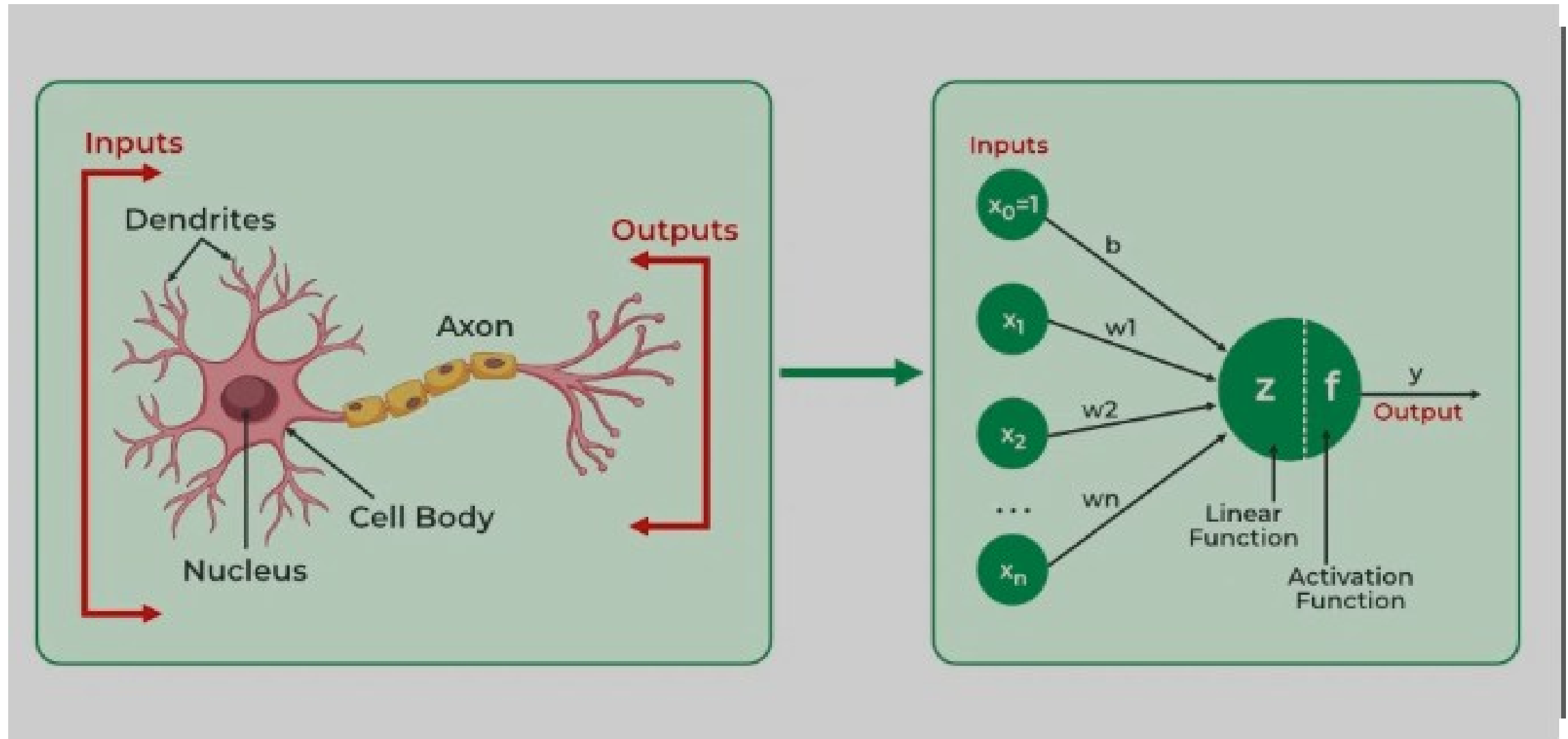
Neural Networks

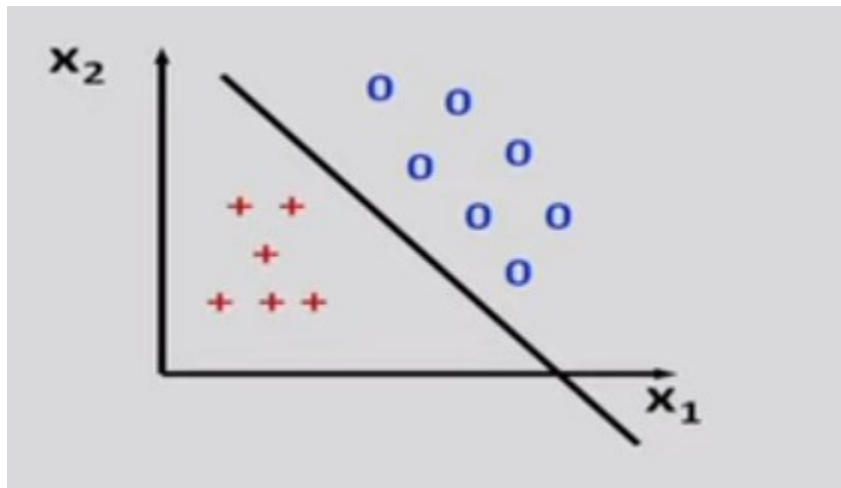
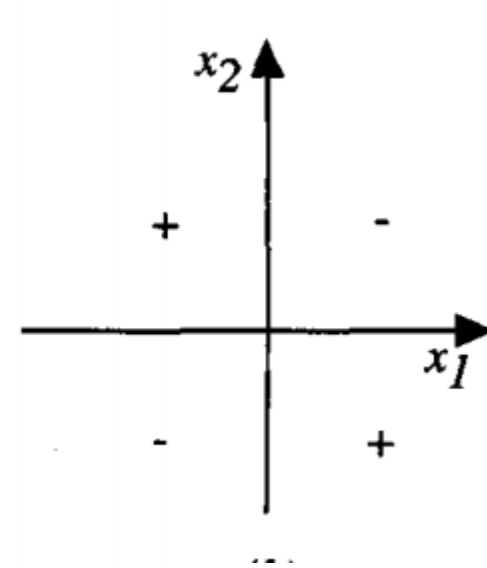
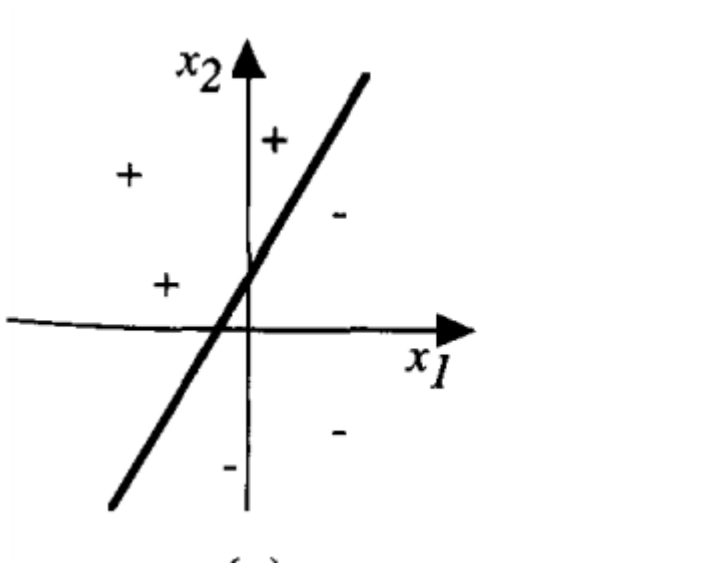
- **Neural Networks** are networks of interconnected neurons, for example in human brains.
- **Artificial Neural Networks** are highly connected to other neurons, and performs computations by combining signals from other neurons.



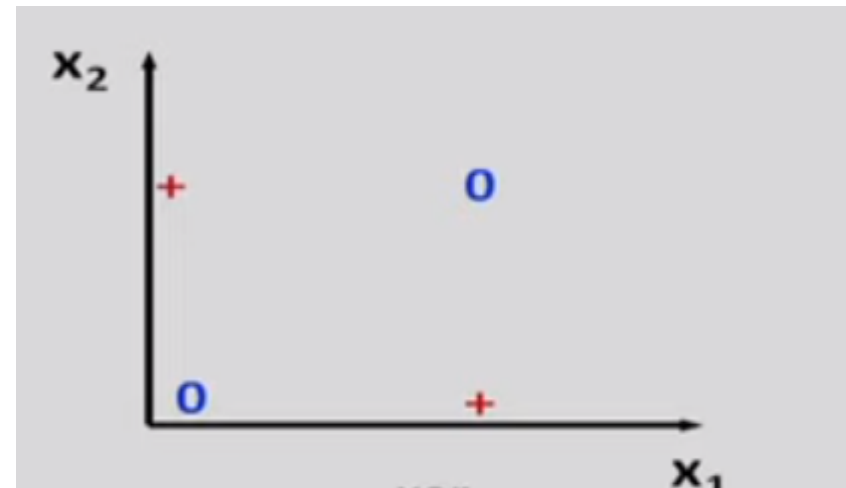
- Outputs of these computations may be transmitted to one or more other neurons.
- The neurons are connected together in a specific way to perform a particular task.

Biological Neural Network (BNN) vs Artificial Neural Network (ANN)



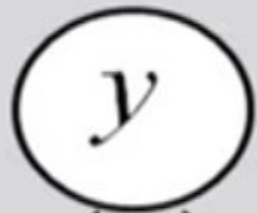


Classification



Regression

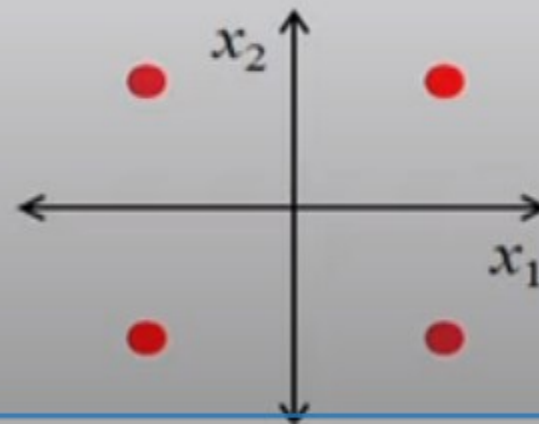
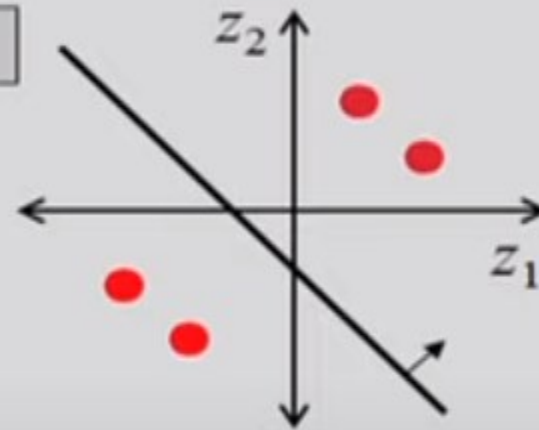
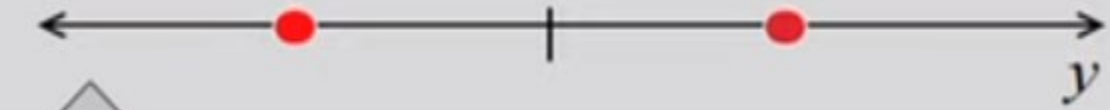
output layer



hidden layer



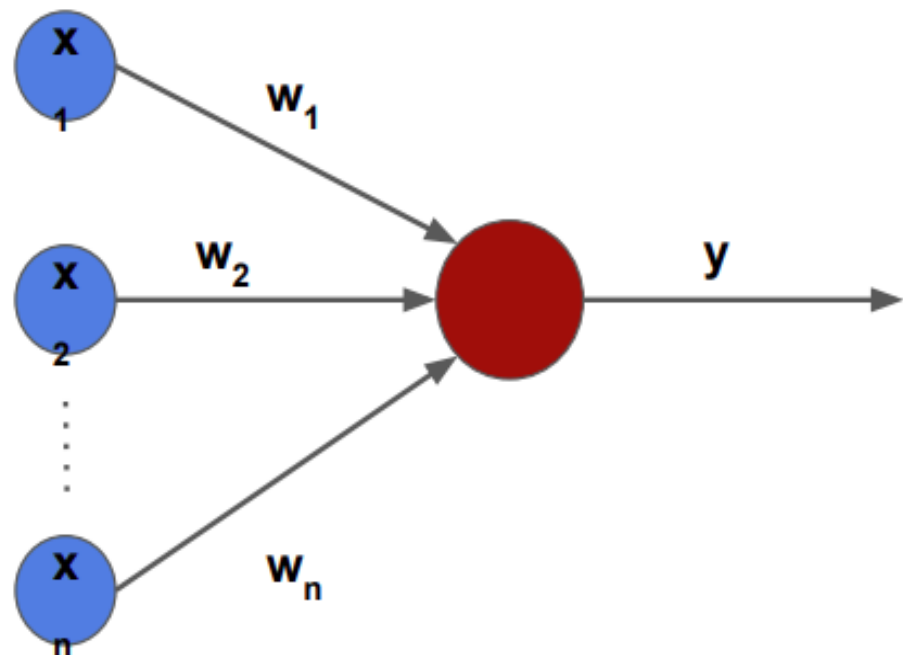
input layer



The Perceptron Model

- Motivated by the biological neuron.
- A perceptron is a computing element where inputs are associated with the weights and the cell having a threshold value.

$$y = \begin{cases} 1, & \text{if } \sum w_i x_i > \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$



Multilayer NN

Perceptron & Sigmoid unit

- Perceptron computes linear combination of inputs and applies a threshold to output
- Sigmoid unit is like a perceptron, which is differentiable threshold unit

Sigmoid / Logistic Function

$$\text{logistic}(u) \equiv \frac{1}{1 + e^{-u}}$$

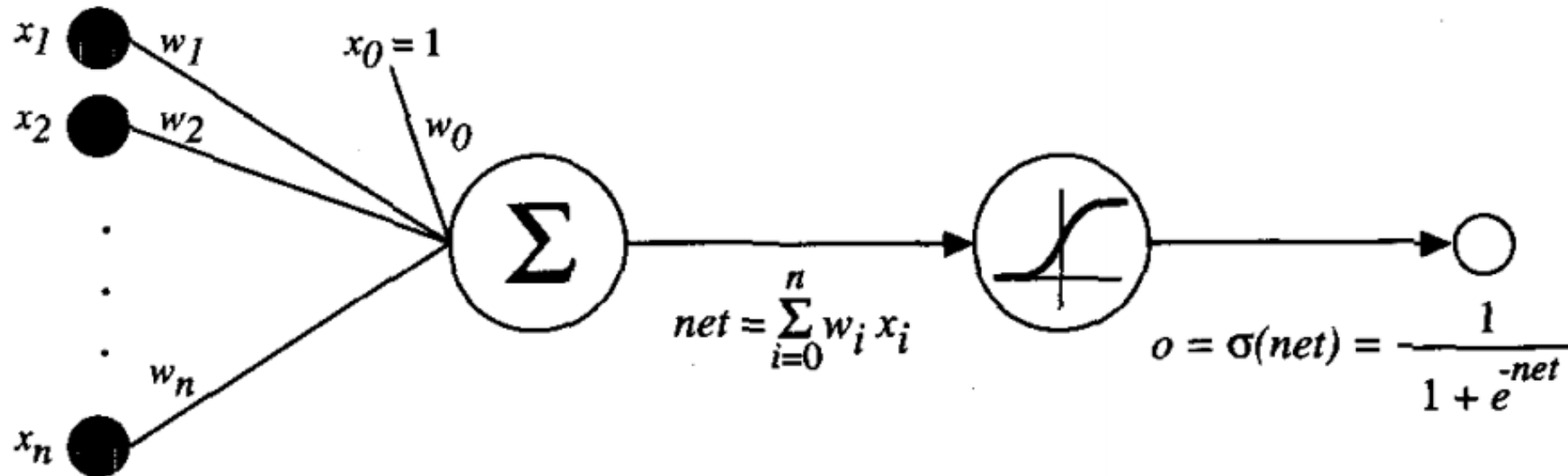
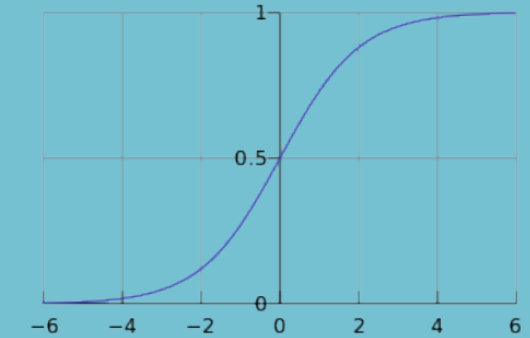
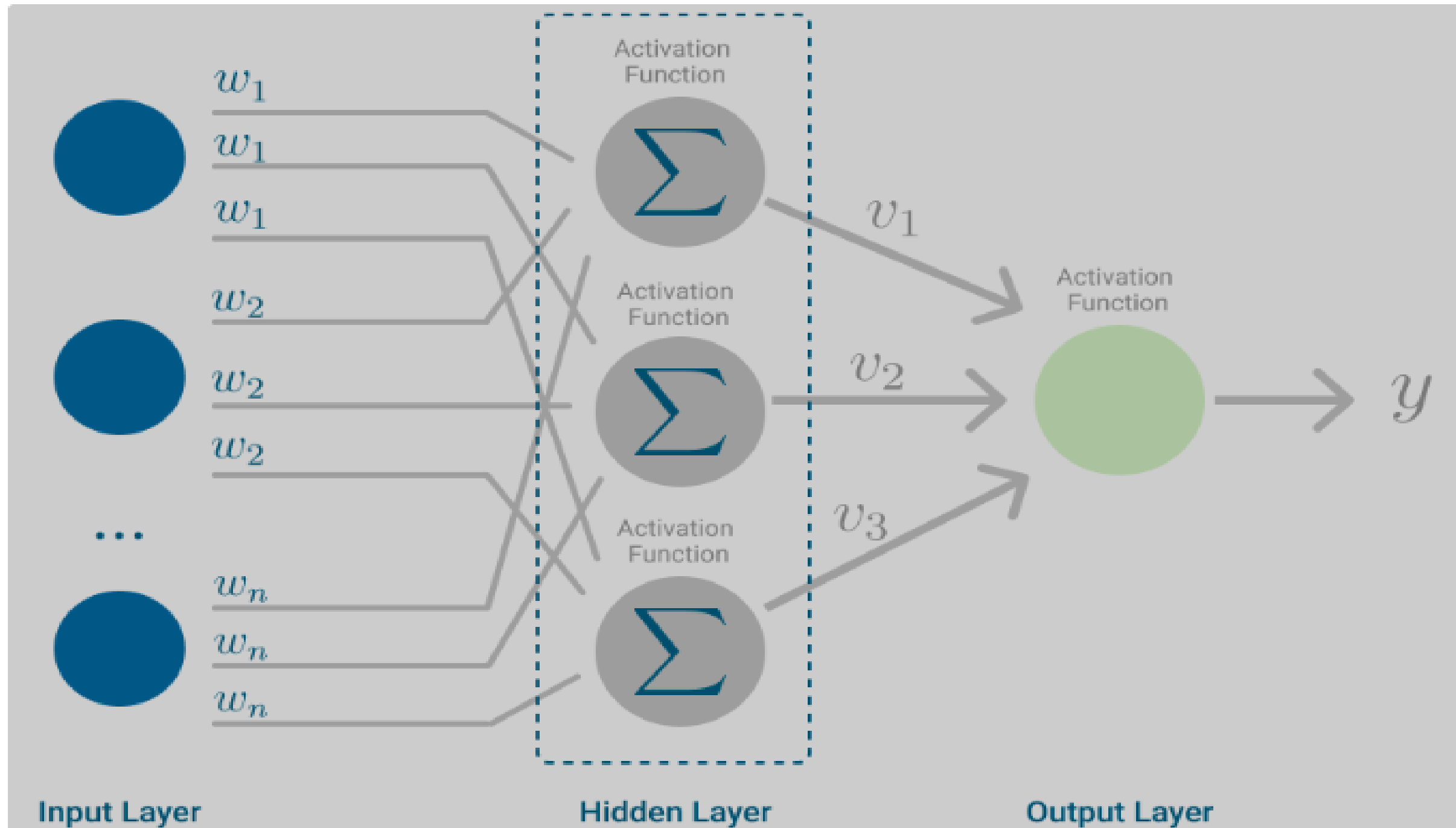
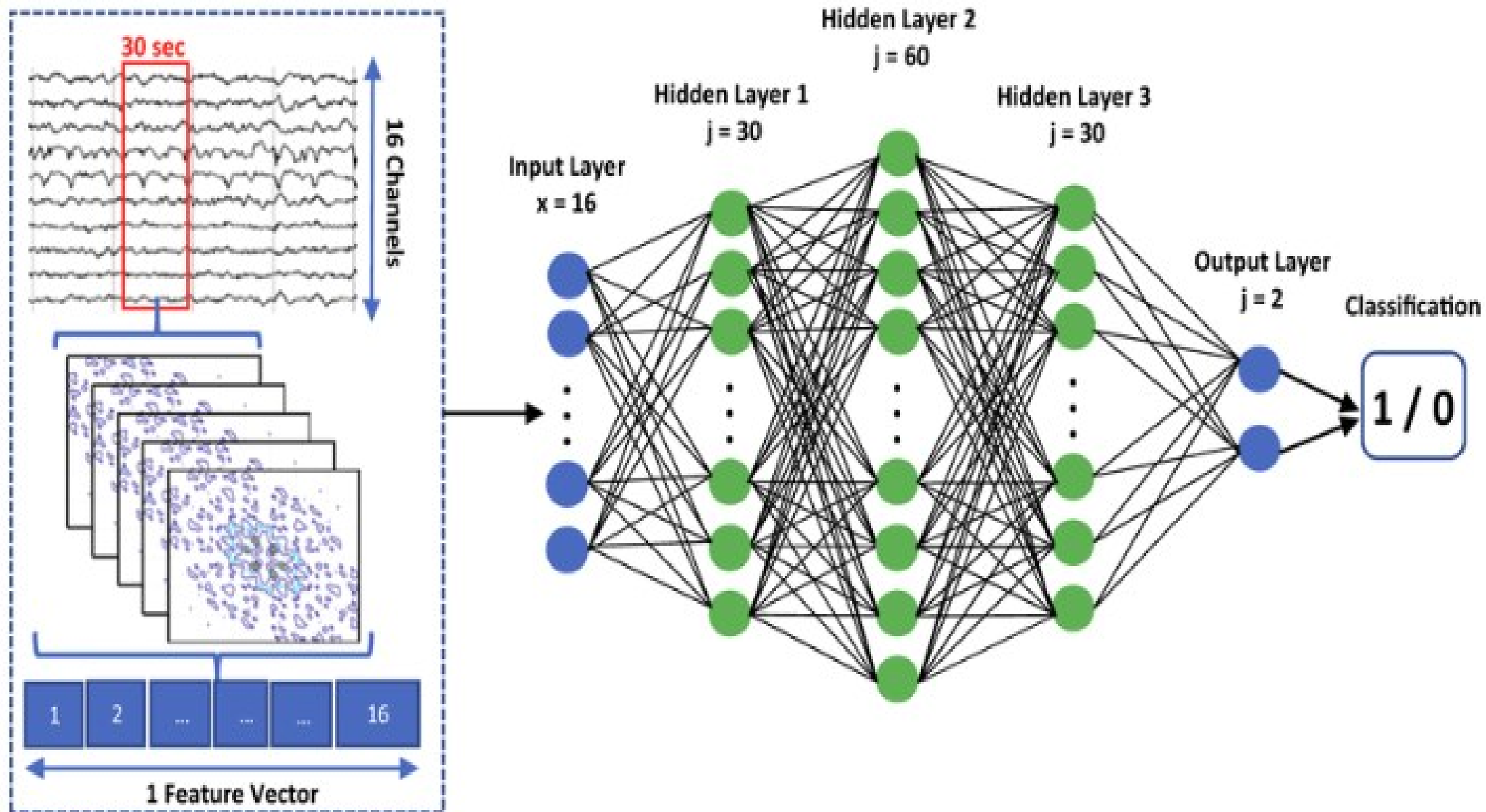


FIGURE 4.6

The sigmoid threshold unit.





Backpropagation Algorithm

- Backpropagation algorithm is used to train artificial neural networks, it can update the weights very efficiently.
- It is a computationally efficient approach to compute the derivatives of a complex cost function.
- Goal is to use those derivatives to learn the weight coefficients for parameterizing a multi-layer artificial neural network.
- It compute the gradient of a cost function with respect to all the weights in the network, so that the gradient is fed to the gradient descent method which in turn uses it to update the weights in order to minimize the cost function.

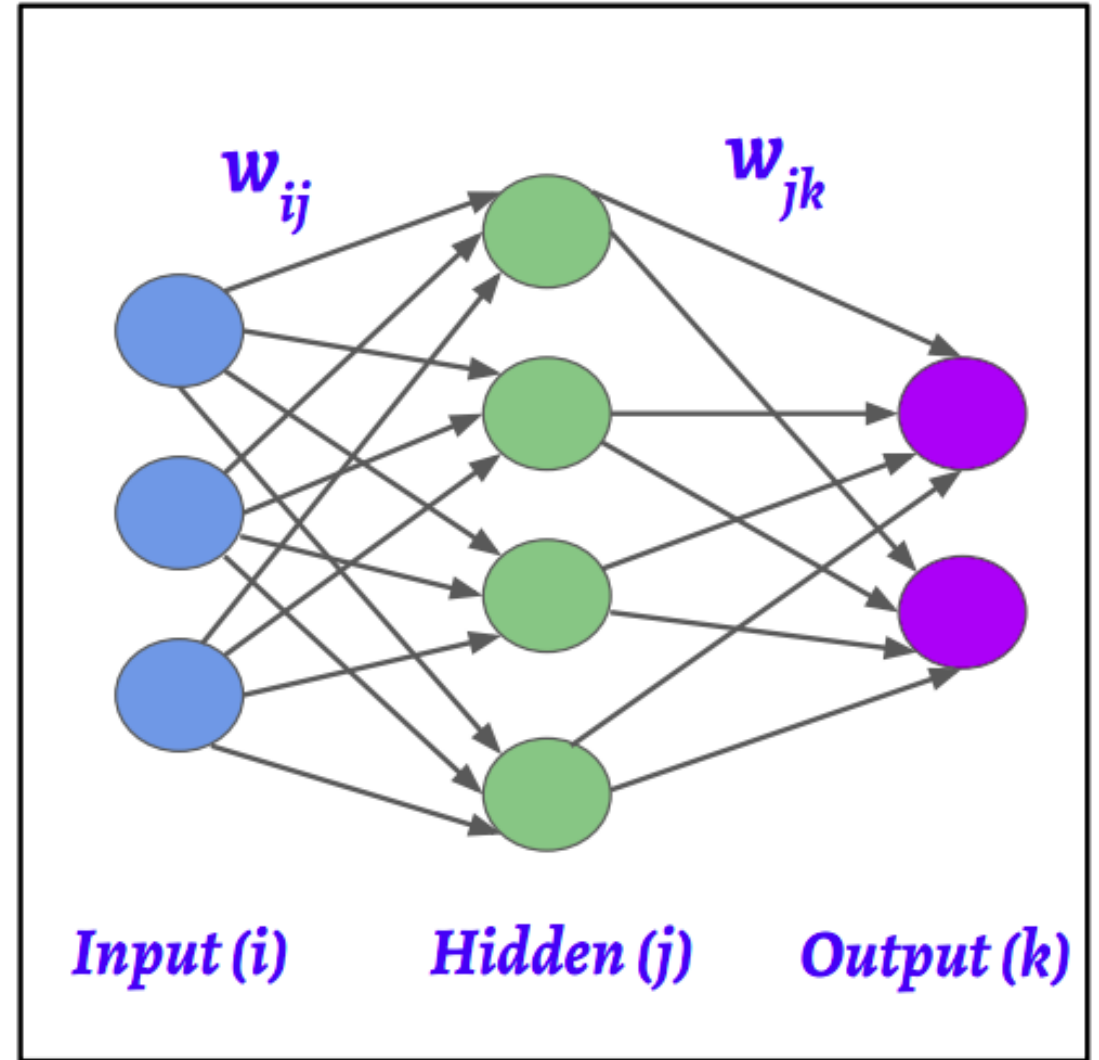
- The total error in the network for a single input is given by the following equation

$$E = \frac{1}{2} \sum_{k=1}^K (a_k - t_k)^2$$

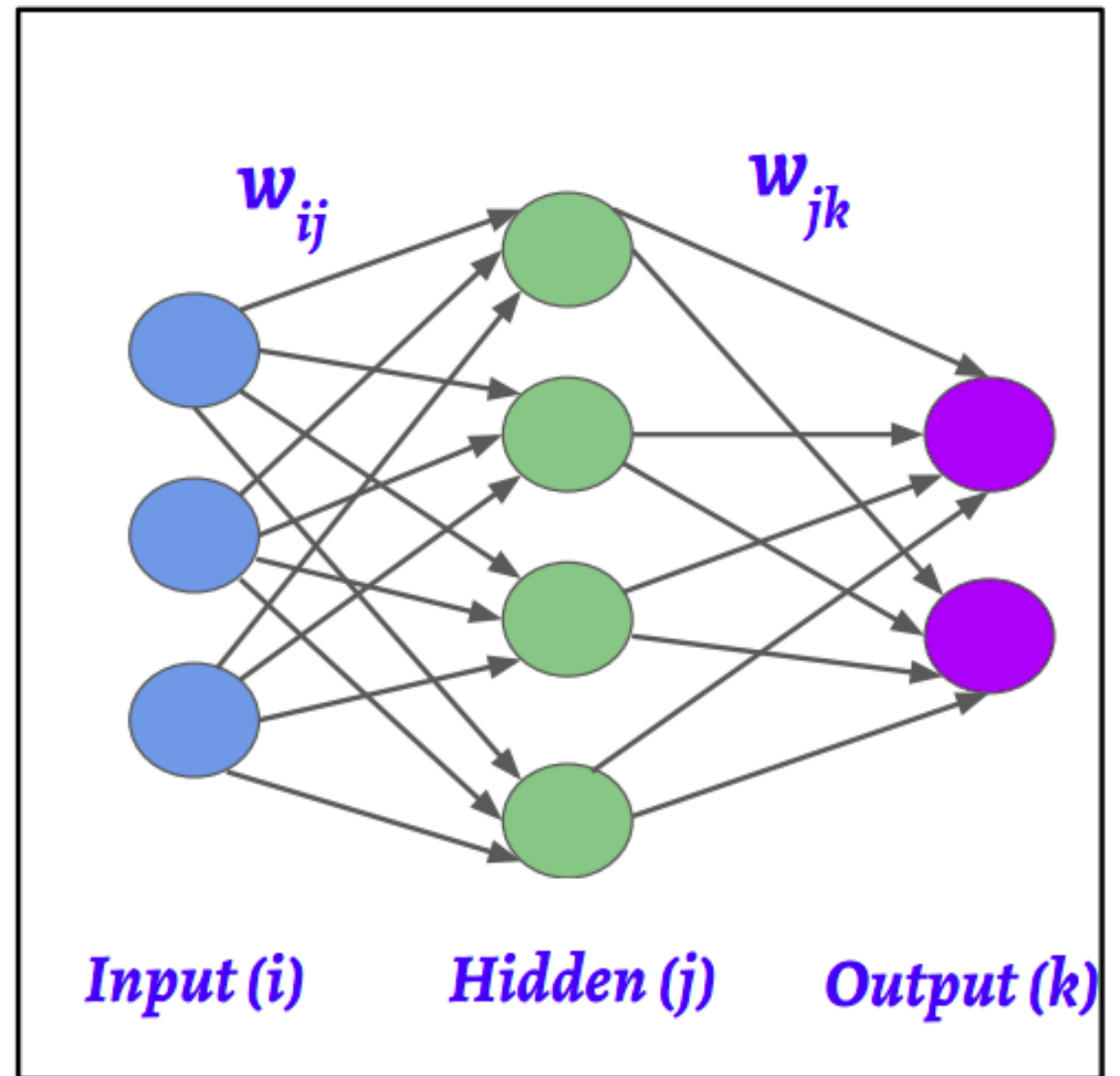
where

a_k : predicted output/activation of a node k

t_k : actual output of a node k



- There are two sets of weights in our network:
 - w_{ij} : from the input to the hidden layer.
 - w_{jk} : from the hidden to the output layer.
- We want to adjust the network's weights to reduce this overall error.
 - $\Delta W \propto -\frac{\partial E}{\partial W}$



According to delta & Gradient principles, weight tuning is done by using

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

The error function can be defined as loss function for all the outputs (may be at hidden layer or output layer)

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

So, The change of error with weight change can be defined as (Using Chain Rule)

$$\begin{aligned} \frac{\partial E_d}{\partial w_{ji}} &= \frac{\partial E_d}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ji}} \\ &= \frac{\partial E_d}{\partial \text{net}_j} x_{ji} \end{aligned}$$

where 'j' is output or hidden

- x_{ji} = the i th input to unit j
- w_{ji} = the weight associated with the i th input to unit j
- $net_j = \sum_i w_{ji}x_{ji}$ (the weighted sum of inputs for unit j)
- o_j = the output computed by unit j
- t_j = the target output for unit j
- σ = the sigmoid function
- $outputs$ = the set of units in the final layer of the network
- $Downstream(j)$ = the set of units whose immediate inputs include the output of unit j

Back Propagation

Case 1: If 'j' is output unit

As it is multi-layer, change of total error estimated at output as well as hidden also

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$\begin{aligned} \frac{\partial E_d}{\partial w_{ji}} &= \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} \\ &= \frac{\partial E_d}{\partial net_j} x_{ji} \end{aligned}$$

Change of error with output is,

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

Lets define 'k' as 'j' i.e jth output

$$\begin{aligned} \frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 \\ &= \frac{1}{2} 2(t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j} \\ &= -(t_j - o_j) \end{aligned}$$

Change of output with net output from preceding node

$$\begin{aligned}\frac{\partial o_j}{\partial net_j} &= \frac{\partial \sigma(net_j)}{\partial net_j} \\ &= o_j(1 - o_j)\end{aligned}$$

$$\begin{aligned}\frac{\partial \sigma}{\partial net_j} &= \frac{\partial}{\partial net_j} \left(\frac{1}{1 + e^{-y}} \right) \\ &= \frac{(1 + e^{-y})(0) - 1(0 + e^{-y})}{(1 + e^{-y})^2} \\ &= \frac{e^{-y}}{(1 + e^{-y})^2} \\ o_j(1 - o_j) &= \frac{1}{1 + e^{-y}} \left[1 - \frac{1}{1 + e^{-y}} \right] \\ &= \frac{1}{1 + e^{-y}} \left[\frac{1 + e^{-y} - 1}{1 + e^{-y}} \right] \\ &= \frac{e^{-y}}{(1 + e^{-y})^2}\end{aligned}$$

Change of output with net output from preceding node

$$\begin{aligned}\frac{\partial o_j}{\partial net_j} &= \frac{\partial \sigma(net_j)}{\partial net_j} \\ &= o_j(1 - o_j)\end{aligned}$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j) o_j(1 - o_j)$$

Therefore,

$$\begin{aligned}\frac{\partial E_d}{\partial w_{ji}} &= \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} \\ &= \frac{\partial E_d}{\partial net_j} x_{ji}\end{aligned}$$

So, according to gradient rule, change in weight can be written as

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j(1 - o_j)x_{ji}$$

Case 2: If 'J' is hidden unit

$$\begin{aligned}\frac{\partial E_d}{\partial net_j} &= \sum_{k \in \text{Downstream}(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} \\&= \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial net_k}{\partial net_j} \\&= \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j} \\&= \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} \frac{\partial o_j}{\partial net_j} \\&= \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} o_j (1 - o_j)\end{aligned}$$

$$\begin{aligned}\frac{\partial E_d}{\partial w_{ji}} &= \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} \\&= \frac{\partial E_d}{\partial net_j} x_{ji}\end{aligned}$$

Rearranging terms and using δ_j to denote $-\frac{\partial E_d}{\partial net_j}$, we have

$$\delta_j = o_j(1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj}$$

and

$$\Delta w_{ji} = \eta \delta_j x_{ji}$$

